

MIP - Mainframe Intelligence Platform

Vision

Enable organizations to understand, evolve, modernize, and transform large-scale legacy mainframe systems with confidence, speed, and reduced risk.

The platform should become the system of intelligence for legacy applications, providing deep visibility into application structure, business rules, dependencies, data lineage, operational behavior, and modernization opportunities.

Problem Statement

Large enterprises across banking, financial services, insurance, healthcare, telecommunications, government, and retail continue to rely on mission-critical mainframe applications developed over decades.

These systems are often:

- Business critical
- Operationally stable
- High performance
- Highly scalable
- Deeply integrated into enterprise operations

However, organizations face significant challenges:

Knowledge Attrition

The engineers, architects, and subject matter experts who originally designed and maintained these systems are retiring or no longer available.

Critical business knowledge remains hidden inside:

- COBOL programs
- JCL
- Copybooks
- CICS transactions
- DB2 schemas
- VSAM datasets
- Scheduler definitions

Monolithic Complexity

Applications have evolved over decades into tightly coupled ecosystems consisting of:

- Thousands of programs

- Thousands of jobs
- Large numbers of shared structures
- Complex batch and online flows

Understanding dependencies becomes difficult.

Slow Change Velocity

Even small changes require significant effort because teams must determine:

- What code is involved
- What business rules exist
- What systems are impacted
- What data is affected

This increases:

- Delivery time
- Operational risk
- Cost

Modernization Risk

Organizations want to modernize but struggle to answer:

- What should be modernized first?
 - What business capabilities exist?
 - What services should be extracted?
 - What dependencies must be preserved?
 - What regression testing is required?
-

What Is Not The Problem

Mainframes are not inherently bad technology.

In fact, they continue to provide:

- Exceptional throughput
- Predictable latency
- Reliability
- Transaction integrity
- Operational maturity
- Horizontal business scalability

The challenge is not performance.

The challenge is understanding and evolving the system safely.

Mission

Transform legacy application understanding from a human-dependent activity into a continuously updated intelligence capability.

Users

Primary Users

Application Developers

Need to understand:

- Program flows
- Dependencies
- Business rules

Production Support Teams

Need to understand:

- Failures
- Batch jobs
- Operational impacts

Architects

Need to understand:

- System structure
- Boundaries
- Modernization opportunities

QA Teams

Need to understand:

- Impacted functionality
- Regression scope

Business Analysts

Need to understand:

- Business processes
- Functional flows

Modernization Teams

Need to understand:

- Service candidates
 - Migration strategy
 - Transformation dependencies
-

Enterprise Scale Assumptions

The platform must support environments containing:

- Large numbers of COBOL programs
- Large numbers of JCL artifacts
- Large numbers of Copybooks
- Large numbers of DB2 objects
- Large numbers of CICS transactions
- Large numbers of scheduler definitions

The architecture should assume enterprise-scale repositories rather than project-scale repositories.

Core Use Cases

Discovery

What exists?

Examples:

- Show all programs.
 - Show all jobs.
 - Show all copybooks.
-

Dependency Analysis

How are components connected?

Examples:

- Who calls this program?
 - Which jobs execute this module?
 - Which copybooks are used?
-

Data Lineage

How does data move?

Examples:

- Which programs update CUSTOMER_ACCOUNT?
 - Which jobs populate this table?
-

Root Cause Analysis

What caused a failure?

Examples:

- Trace failed batch execution.
 - Identify impacted downstream systems.
-

Impact Analysis

What breaks if something changes?

Examples:

- Table change impact.
 - Program change impact.
 - Copybook change impact.
-

Knowledge Transfer

Help new engineers become productive faster.

Examples:

- Explain program in plain English.
 - Explain business flow.
 - Generate documentation.
-

Modernization Planning

Support modernization decisions.

Examples:

- Identify service candidates.
 - Detect bounded contexts.
 - Identify tightly coupled domains.
 - Generate modernization roadmap.
-

Product Philosophy

The platform should not start with AI.

The platform should start with facts.

Source Code ↓ Metadata ↓ Knowledge Graph ↓ Reasoning ↓ AI

AI should consume knowledge.

AI should not replace knowledge.

Long-Term Vision

A user should be able to ask:

"How does payment posting work?"

and receive:

- Business capability
- Jobs involved
- Programs involved
- Tables involved
- Call hierarchy
- Business rules
- Impact analysis
- Modernization recommendations

without requiring tribal knowledge or SME intervention.

Success Criteria

The platform successfully reduces:

- Dependency on legacy experts
- Time required to understand applications

- Time required for impact analysis
- Modernization risk
- Operational knowledge gaps

while increasing:

- Developer productivity
- System visibility
- Change confidence
- Modernization readiness
- Organizational knowledge retention